1. Представьте, что вы устроились работать Дата-инженером в некоторый интернет магазин. До вас в этой компании уже работал один разработчик, который придумал небольшую базу данных для этого магазина и потом неожиданно уволился. Ваша задача провести проверку существующей архитектуры и решить корректна ли она илиможно внести некоторые доработки.
   Вы обратили внимание на таблицу с информацией о зарегистрировавшихся покупателях (клиентах).

Таблица имеет следующую схему:

```
clients (
        client_id          number,  --уникальный id клиента
        client_name        varchar(255), --имя клиента
        client_surname     varchar(255),  --фамилия клиента
        login              varchar(30),  --логин, который придумал клиент
        city_id            number,  --id города, который указал клиент (в интерфейсе выбирается название города, а в таблицу сохраняется id)
        age                number,  --возраст клиента
        reg_date           date     –дата регистрации на сайте
);
```

Считаете ли вы данный набор и смысл полей корректным? Если нет, то напишите что повашему мнению некорректно и какие изменения внесли бы.

Ответ:

Возраст нужно хранить не как number, а как date т.к. это значение, которое меняется со временем.

Добавить PK на client_id, Unique на login, на city_id foreign key, также можно отчество добавить, для историчности и аналитики можно добавить поля load_dttm, updated_at, is_active

2. Представьте, что вы устроились работать Дата-инженером в некоторый интернет магазин. До вас в этой компании уже работал один разработчик, который придумал небольшую базу данных для этого магазина и потом неожиданно уволился. Ваша задача провести проверку существующей архитектуры и решить корректна ли она илиможно внести некоторые доработки.
Вы обратили внимание на таблицу с информацией о товарах магазина.

Таблица имеет следующую схему:

```
items (
        item_id        number, --уникальный id товара
        item_name      varchar(255), --наименование товара
        item_cost      number –стоимость товара
);
```

Глядя на эту таблицу вы вспоминаете, что мы живём в рыночной экономике и цена, если не каждый день, то каждый месяц может меняться. К примеру, сезонные повышения цен или наоборот скидки (перед началом учебного года или новым годом). Так же вы знаете что у магазина есть отдел финансовой отчётности, в котором сотрудникам нужно строить различныотчёты (о доходах например) за разные промежутки времени, в том числе и за прошлые месяцы и даже прошлые года.

Как бы вы доработали архитектуру таблицы, что бы обеспечить историческое хранениестоимости товара?

Тут проблема историчности, можно разделить на таблицу справочник

Я бы вынес стоимость в отдельную таблицу истории цен с датами начала и окончания действия. Это позволит корректно строить финансовые отчёты за прошлые периоды.

Items (item_id, item_name) и Таблица истории цен

item_prices (

```
    item_id       BIGINT NOT NULL,
    price         DECIMAL(10,2) NOT NULL,
    valid_from    DATE NOT NULL,
    valid_to      DATE,
    is_current    BOOLEAN,
    PRIMARY KEY (item_id, valid_from)
);
```

3. Требуется проверить запрос на корректность и исправить там ошибки, если они есть. Существует учебная схема HR, содержащая таблицы: employees, departments и locations Необходимо получить все отделы0, расположенные в Сеуле в которых все сотрудники, не имеющие менеджера, зарабатывают в общей сложности более 100000.
Для решения задачи был написан следующий запрос:

```
SELECT DEPARTMENT_ID,
     DEPARTMENT_NAME,
     SUM(SALARY) TOTAL_SALARY
  FROM EMPLOYEE E,
     DEPARTMENTS D
WHERE E.DEPARTMENT_ID = D.DEPARTMENT_ID
  AND MANAGER_ID = NULL
  AND LOCATION_ID = (SELECT LOCATION_ID
                     FROM LOCATIONS
                    WHERE CITY = 'SEOUL')
  AND SUM(SALARY) >= 100000
GROUP BY DEPARTMENT_NAME
```

1. Проверка на NULL
2. Агрегация в where, а не having
3. В group by не все поля выборки
4. Неявный JOIN
5. Подзапрос может несколько строк вернуть

Исправленный запрос:

```
SELECT
    d.department_id,
    d.department_name,
    SUM(e.salary) AS total_salary
FROM employees e
JOIN departments d
  ON e.department_id = d.department_id
JOIN locations l
  ON d.location_id = l.location_id
WHERE e.manager_id IS NULL
  AND l.city = 'SEOUL'
```

```
GROUP BY
    d.department_id,
    d.department_name
HAVING SUM(e.salary) > 100000;
```

4.

4. В некоторой комнате на пол уронили карандаш. Объясните почему вы не можете через него перепрыгнуть?

Высота карандаша ≈ 0, следовательно, порог препятствия равен 0. Любой прыжок через 0 эквивалентен отсутствию прыжка, то есть перепрыгнуть - значит ничего не сделать.